Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к практической работе №2
на тему:
«ЗАЩИЩЁННЫЙ РЕЖИМ РАБОТЫ ПРОЦЕССОРА PENTIUM»
БГУИР 6-05-0612-02 01

Выполнил студент группы 453503
МИХАЛКО Александр Николаевич

_____
(дата, подпись студента)

Проверил сотрудник каф. Информатики
МАКСИМЧИКОВ АРТЁМ ВЯЧЕСЛАВОВИЧ

_____
(дата, подпись преподавателя)

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

Цель работы: изучить особенности защищённого режима процессора. Получить практические навыки по программированию переключения процессора из реального в защищённый режим и обратно.

Задание: написать программу, переключающую процессор в защищённый режим, выводящую на экране сообщение и затем возвращающую процессор в реальный режим. Вариант не требуется.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Программа написана на языке ассемблера для процессоров *i386* и выше (синтаксис *NASM*) и собирается в *COM*-файл командой *nasm -f bin pm.asm -o pm.com*. Программа должна запускаться в реальном режиме без менеджеров памяти (*EMM386*), так как под их управлением процессор работает в режиме виртуального *8086* и самостоятельное переключение режимов невозможно. Поэтому в начале программы командой *smsw* считывается младшее слово регистра *CR0* и проверяется бит *PE*. Если он установлен, выводится сообщение об ошибке и программа завершается.

Для работы в защищённом режиме создана глобальная дескрипторная таблица *GDT*, состоящая из четырёх дескрипторов (таблица 1). Базовые адреса сегментов кода и данных вычисляются во время выполнения как *CS*·16 и записываются в соответствующие дескрипторы. Дескриптор видеопамяти имеет базу *B8000h* и предел *0FFFh*, что позволяет выводить символы напрямую в текстовый видеобуфер. Дескриптор данных удовлетворяет требованиям к дескрипторам для возврата в реальный режим: предел 64 Кбайт (*FFFFh*), байтовая гранулярность ($G = 0$), разрешена запись ($W = 1$), сегмент присутствует в памяти ($P = 1$).

Таблица 1 – Дескрипторы глобальной дескрипторной таблицы

Селектор	**Назначение**	**База**	**Предел**	**Байт доступа**
00h	Нулевой дескриптор	0	0	00h
08h	Сегмент кода	CS·16	FFFFh	9Ah
10h	Видеопамять	B8000h	0FFFh	92h
18h	Сегмент данных	CS·16	FFFFh	92h

Образ регистра *GDTR* (16-разрядный предел и 32-разрядный линейный адрес таблицы) формируется в области памяти *gdtr*. Линейный адрес таблицы вычисляется как *CS*·16 плюс смещение метки *gdt*.

Переключение в защищённый режим выполняется в следующем порядке. Сохраняются указатель стека *SP* и сегмент возврата *CS* (в переменных *save_sp* и *rm_ptr*). Командой *cli* запрещаются маскируемые прерывания, а записью значения *80h* в порт *70h* – немаскируемые. Командой

lgdt загружается регистр *GDTR*, после чего в регистре *CR0* устанавливается бит *PE*. Сразу после этого выполняется команда дальнего перехода *far jmp*, закодированная байтом *0EAh*, смещением и селектором кода *08h*. Она очищает конвейер команд процессора и загружает в регистр *CS* селектор дескриптора кодового сегмента.

В защищённом режиме в регистры *DS* и *SS* загружается селектор данных *18h*, а в регистр *ES* – селектор видеопамяти *10h*. Сообщение *Hello from Protected Mode!* выводится в цикле командами *lodsb* и *stosw* прямо в видеопамять, начиная со строки 12 и столбца 20, с атрибутом *1Eh* (жёлтые символы на синем фоне).

Для возврата в реальный режим во все сегментные регистры (*DS*, *ES*, *SS*, *FS*, *GS*) загружается селектор *18h*, дескриптор которого содержит «реальные» атрибуты. Затем сбрасывается бит *PE* регистра *CR0*, и дальним переходом через сохранённый указатель *rm_ptr* в регистр *CS* загружается сегментный адрес реального режима. После этого восстанавливаются регистры *DS*, *ES*, *SS*, *SP*, обнуляются *FS* и *GS*, разрешаются немаскируемые (запись *0* в порт *70h*) и маскируемые (команда *sti*) прерывания. В завершение выводится сообщение о возврате в реальный режим, ожидается нажатие клавиши и программа завершается.

Текст программы представлен на листинге 1.

Листинг 1 – Программа переключения процессора из реального режима в защищённый и обратно

```
        cpu     386
        bits    16
        org     100h

start:
        smsw    ax
        test    al, 1
        jz      real_ok
        mov     dx, msg_v86
        mov     ah, 9
        int     21h
        mov     ax, 4C01h
        int     21h

real_ok:
        mov     dx, msg_start
        mov     ah, 9
        int     21h

        xor     eax, eax
        mov     ax, cs
        shl     eax, 4
        mov     [gdt_code+2], ax
        mov     [gdt_data+2], ax
        shr     eax, 16
        mov     [gdt_code+4], al
        mov     [gdt_data+4], al

        xor     eax, eax
```

```
        mov     ax, cs
        shl     eax, 4
        add     eax, gdt
        mov     [gdtr+2], eax
        mov     word [gdtr], gdt_end - gdt - 1

        mov     [save_sp], sp
        mov     [rm_ptr+2], cs

        cli
        mov     al, 80h
        out     70h, al
        lgdt    [gdtr]

        mov     eax, cr0
        or      al, 1
        mov     cr0, eax

        db      0EAh
        dw      pm_entry
        dw      CODE_SEL

pm_entry:
        mov     ax, DATA_SEL
        mov     ds, ax
        mov     ss, ax
        mov     ax, VIDEO_SEL
        mov     es, ax

        mov     si, pm_text
        mov     di, (12*80 + 20) * 2
        mov     ah, 1Eh
.print:
        lodsb
        or      al, al
        jz      .done
        stosw
        jmp     .print
.done:

        mov     ax, DATA_SEL
        mov     ds, ax
        mov     es, ax
        mov     ss, ax
        mov     fs, ax
        mov     gs, ax

        mov     eax, cr0
        and     al, 0FEh
        mov     cr0, eax

        jmp     far [cs:rm_ptr]

back_to_real:
        mov     ax, cs
        mov     ds, ax
        mov     es, ax
        mov     ss, ax
        mov     sp, [save_sp]
        xor     ax, ax
        mov     fs, ax
        mov     gs, ax
```

```
        mov     al, 0
        out     70h, al
        sti

        mov     dx, msg_back
        mov     ah, 9
        int     21h
        mov     ah, 1
        int     21h
        mov     ax, 4C00h
        int     21h

CODE_SEL  equ 08h
VIDEO_SEL equ 10h
DATA_SEL  equ 18h

        align   8
gdt:
        dq      0
gdt_code:
        dw      0FFFFh, 0
        db      0, 9Ah, 0, 0
gdt_video:
        dw      0FFFh, 8000h
        db      0Bh, 92h, 0, 0
gdt_data:
        dw      0FFFFh, 0
        db      0, 92h, 0, 0
gdt_end:

gdtr:   dw      0
        dd      0

rm_ptr: dw      back_to_real, 0
save_sp dw      0

pm_text   db 'Hello from Protected Mode!', 0
msg_start db 'Real mode. Switching to protected mode...', 13, 10, '$'
msg_back  db 13, 10, 13, 10, 'Back in real mode. Press any key...$'
msg_v86   db 'CPU already in V86/protected mode. Run without EMM386.',
13, 10, '$'
```

Результат работы программы представлен на рисунке 1. Сначала выводится сообщение реального режима, затем, после переключения в защищённый режим, в строке 12 экрана появляется сообщение, записанное напрямую в видеопамять, после чего процессор возвращается в реальный режим и выводится соответствующее сообщение.

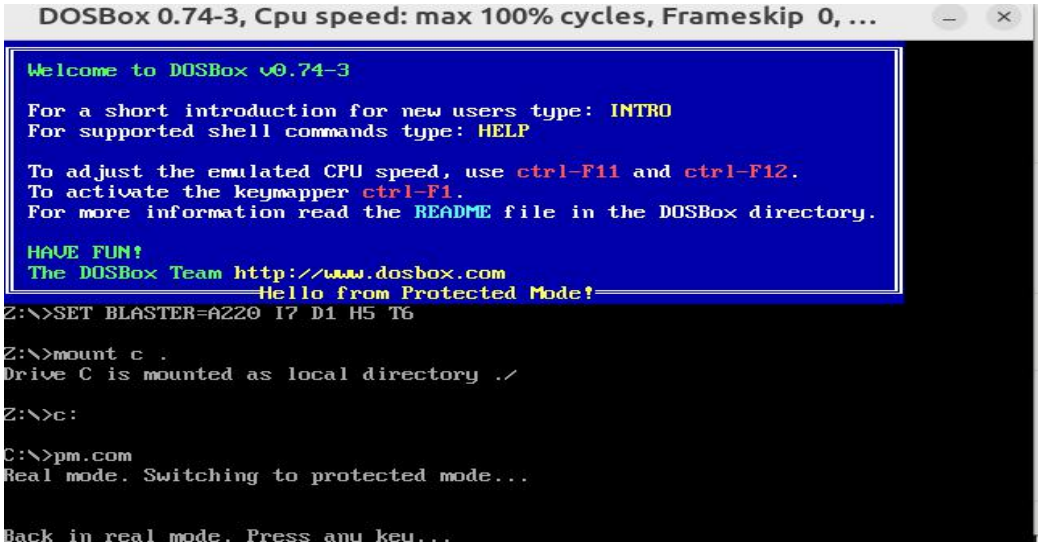


Рисунок 1 – Результат работы программы

Как видно из результатов выполнения, процессор успешно перешёл в защищённый режим, выполнил в нём вывод сообщения через дескриптор видеопамяти и вернулся в реальный режим, после чего работа программы была продолжена штатным образом.

3 ВЫВОД

В ходе выполнения практической работы были изучены особенности защищённого режима процессора *Pentium*: сегментная адресация с помощью селекторов и дескрипторов, структура глобальной дескрипторной таблицы и регистра *GDTR*, а также роль бита *PE* регистра *CR0*.

Получены практические навыки программирования переключения процессора из реального режима в защищённый и обратно. Разработанная программа корректно выполняет все необходимые подготовительные действия, выводит сообщение в защищённом режиме и возвращает процессор в реальный режим с восстановлением состояния системы.